Priority Queue Study Guide - Java

What is a Priority Queue?

A Priority Queue is a data structure where elements are served based on their priority rather than their insertion order. Elements with higher priority are dequeued before elements with lower priority.

Key Characteristics:

- Elements are ordered by priority, not insertion order
- Higher priority elements are processed first
- Implements a heap data structure internally
- Part of Java Collections Framework

Priority Queue in Java

Importing Priority Queue

```java
import java.util.PriorityQueue;
import java.util.Collections;
```

Creating a Priority Queue

Default Priority Queue (Min-Heap)

```java
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Priority Queue with Custom Comparator (Max-Heap)

```java
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
```

Priority Queue with Initial Capacity

```java
PriorityQueue<Integer> pq = new PriorityQueue<>(10);
```

Common Methods

Adding Elements

```java
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(30);
pq.offer(10);  // Same as add()
pq.add(20);
pq.add(5);
```

Removing Elements

```java
// Remove and return the highest priority element
int element = pq.poll();  // Returns null if empty
int element2 = pq.remove();  // Throws exception if empty
```

Accessing Elements

```java
// Peek at the highest priority element without removing
int top = pq.peek();  // Returns null if empty
int top2 = pq.element();  // Throws exception if empty
```

Other Useful Methods

```java
// Check if empty
boolean isEmpty = pq.isEmpty();

// Get size
int size = pq.size();

// Check if contains element
boolean contains = pq.contains(10);

// Clear all elements
pq.clear();
```

Working with Different Data Types

String Priority Queue

```java
PriorityQueue<String> stringPQ = new PriorityQueue<>();
stringPQ.add("banana");
stringPQ.add("apple");
stringPQ.add("cherry");
// Output order: apple, banana, cherry (alphabetical)
```

Custom Objects with Comparable

```java
class Student implements Comparable<Student> {
    String name;
    int grade;

    public Student(String name, int grade) {
        this.name = name;
        this.grade = grade;
    }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.grade, other.grade);
    }
}

PriorityQueue<Student> studentPQ = new PriorityQueue<>();
studentPQ.add(new Student("Alice", 85));
studentPQ.add(new Student("Bob", 92));
```

Custom Comparator

```java
PriorityQueue<Student> pq = new PriorityQueue<>((s1, s2) ->
    Integer.compare(s2.grade, s1.grade)); // Higher grades first
```

Complete Example Programs

Example 1: Basic Priority Queue Operations

```java
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        // Adding elements
        pq.add(30);
        pq.add(10);
        pq.add(20);
        pq.add(5);

        System.out.println("Priority Queue: " + pq);

        // Removing elements in priority order
        while (!pq.isEmpty()) {
            System.out.println("Removed: " + pq.poll());
        }
    }
}
```

Example 2: Max-Heap Implementation

```java
import java.util.PriorityQueue;
import java.util.Collections;

public class MaxHeapExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

        maxHeap.add(10);
        maxHeap.add(30);
        maxHeap.add(20);
        maxHeap.add(5);

        while (!maxHeap.isEmpty()) {
            System.out.println("Max element: " + maxHeap.poll());
        }
    }
}
```

Important Notes

Time Complexity

- **Insertion**: O(log n)
- **Deletion**: O(log n)
- **Peek**: O(1)
- **Search**: O(n)

Key Points to Remember

- Default PriorityQueue is a **min-heap** (smallest element has highest priority)
- Elements must be comparable or provide a custom comparator
- Not thread-safe (use PriorityBlockingQueue for thread safety)
- Does not allow null elements
- Iterator does not guarantee any particular order

Common Pitfalls

- Forgetting that default is min-heap, not max-heap
- Trying to add null elements

- Assuming iterator returns elements in priority order
- Modifying objects after adding them to the queue (can break heap property)

Quiz Questions

1. What is the default ordering of a PriorityQueue in Java? a) FIFO (First In First Out) b) LIFO (Last In First Out) c) Min-heap (smallest element first) d) Max-heap (largest element first)

Answer: c) Min-heap (smallest element first)

2. Which method should you use to remove and return the highest priority element from a PriorityQueue? a) remove() b) poll() c) peek() d) Both a and b

Answer: d) Both a and b

3. What happens when you try to add a null element to a PriorityQueue? a) It gets added successfully b) It throws a NullPointerException c) It gets ignored d) It gets added with lowest priority

Answer: b) It throws a NullPointerException

4. How do you create a max-heap using PriorityQueue? a) new PriorityQueue<>(true) b) new PriorityQueue<>(Collections.reverseOrder()) c) new PriorityQueue<>(MAX_HEAP) d) new PriorityQueue<>().reverse()

Answer: b) new PriorityQueue<>(Collections.reverseOrder())

5. What is the time complexity of inserting an element into a PriorityQueue? a) O(1) b) O(log n) c) O(n) d) O(n log n)

Answer: b) O(log n)

Practice Coding Questions

1. Task Management System Create a program that manages tasks with different priorities. Each task has a name and priority level (1-10, where 10 is highest priority). Implement methods to:

- Add a task
- Execute the highest priority task
- Display all remaining tasks

2. Kth Largest Element Write a method that finds the kth largest element in an array using a PriorityQueue. The method signature should be:

```java
public static int findKthLargest(int[] nums, int k)
```

3. Merge K Sorted Lists Given k sorted integer arrays, write a program using PriorityQueue to merge them into a single sorted array.

4. Student Grade Processor Create a Student class with name and grade fields. Write a program that:

- Reads student data (name and grade)
- Uses PriorityQueue to maintain students sorted by grade (highest first)
- Prints students in order of their grades

5. Hospital Emergency System Design a simple emergency room system where patients are treated based on severity level (1=critical, 2=urgent, 3=normal). Create a Patient class and implement methods to:

- Add patients to the queue
- Treat the next patient (highest priority)
- Display waiting patients